

UFRPE-UAST

Esta é uma marca d'água para a versão de teste, compre para obter a versão completa.

Benefícios para usuários registrados:

1. Converte os documentos inteiros.

2. Nenhuma limitação para os documentos finais.

Celso Brennand

Remover Agora

Tipos de arquitetura

AULA N° 04

Os slides são baseados no livro Sistemas Distribuídos, de TANENBAUM, A. S. e STEEN, M. V.

Na aula anterior, vimos:

- Estilos arquitetônicos e arquiteturas de sistemas

Conteúdo desta aula:

- Arquitetura descentralizada
- Arquitetura híbrida

Arquitetura de sistema - Descentralizadas

Cliente-servidor possuem duas distribuições:

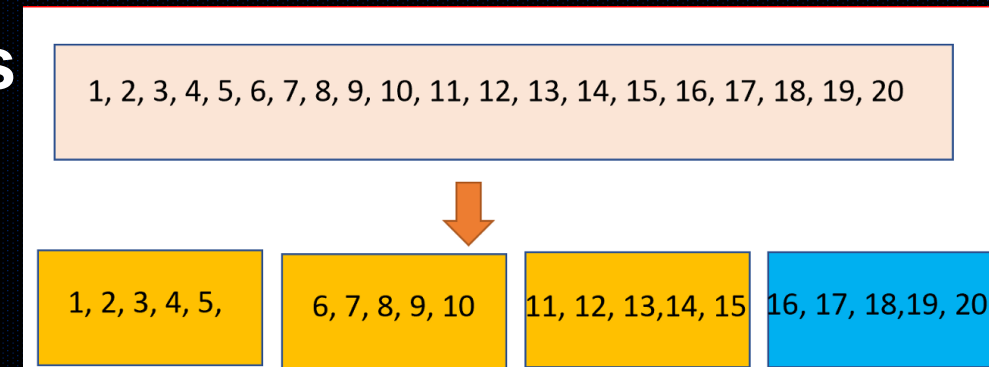
- **Distribuição vertical**
 - Componentes logicamente diferentes em máquinas diferentes
 - Cada máquina vai executar um conjunto específico e pré-determinado de funções
 - As funções de cada um são bem definidas
 - E.g. Servidor de Dados, Servidor de Web...

Arquitetura de sistema - Descentralizadas

Cliente-servidor possuem duas distribuições:

- **Distribuição horizontal**

- Cliente ou servidor pode ser fisicamente subdividido em partes logicamente equivalentes
- Pode possuir porção própria de dados
- Balanceamento de carga
- Papéis não muito bem definidos
- Ex.: Peer-to-Peer que envia um “*pedaço*” de um arquivo



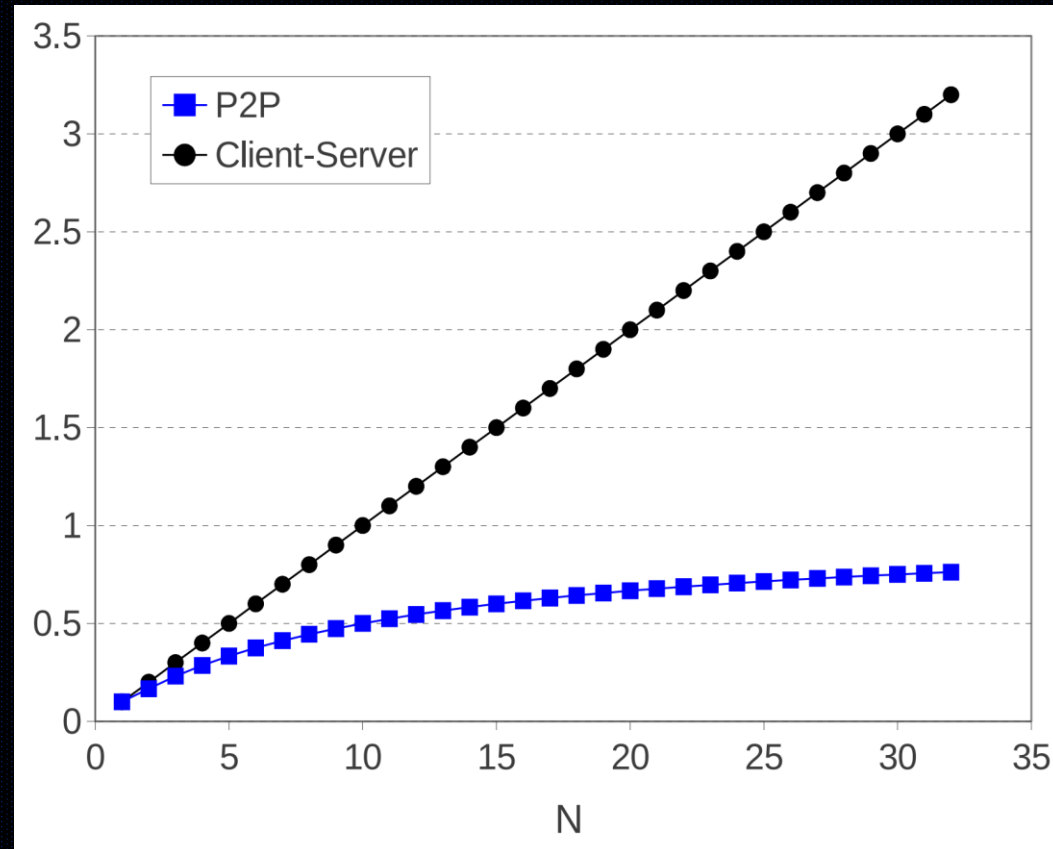
Arquitetura de sistema – Descentralizadas: Peer-to-Peer

- Processos são semelhantes e *flat*
- Cada processo age ao mesmo tempo como cliente e servidor (*servente*) –
- Muito mais “sistema distribuído”, mais difícil
 - ex. intermitência (entra e sai) dos pares
- **Ideia central:** cliente também é servidor
 - demanda gerada por clientes pode ser atendida por outros clientes
- Recursos disponíveis nos clientes usados para prover serviço para outros clientes
 - CPU, disco, banda, etc...
- O cliente é quem inicia a requisição

Arquitetura de sistema – Descentralizadas: Peer-to-Peer

client upload rate = u ,
 $F/u = 1$ hour, $u_s = 10u$,
 $d_{\min} \geq u_s$

- Sistema escalável:
crescimento do tempo
é sublinear em N

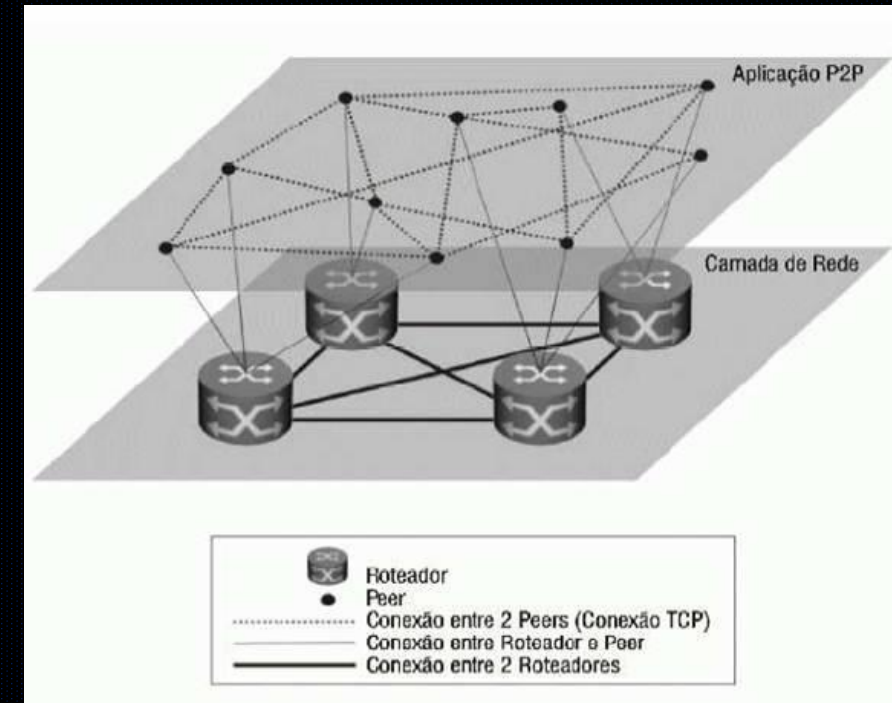


Escalabilidade!

Arquitetura de sistema – Descentralizadas: Peer-to-Peer

Rede de sobreposição: rede onde os nós são formados pelos processos e os enlaces denotam os canais de comunicação.

- Os nós em uma rede de sobreposição são conectados por links virtuais/lógicos, por meio de muitos links físicos na rede subjacente.



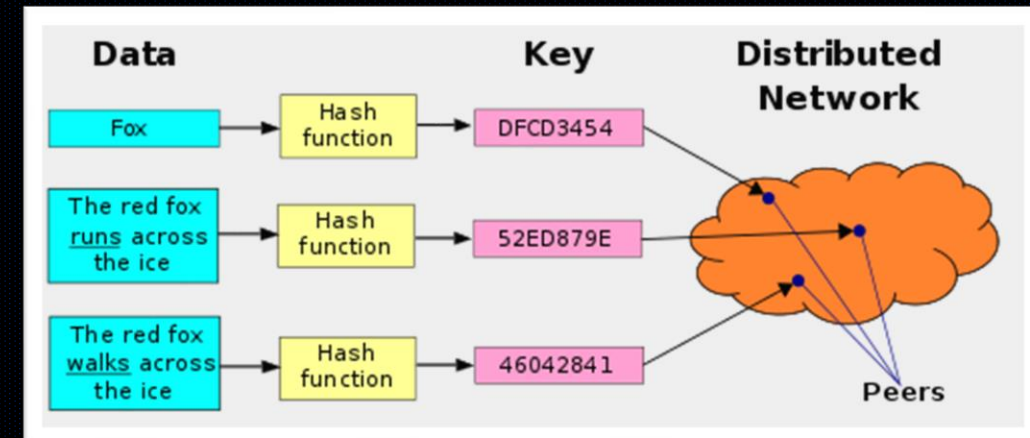
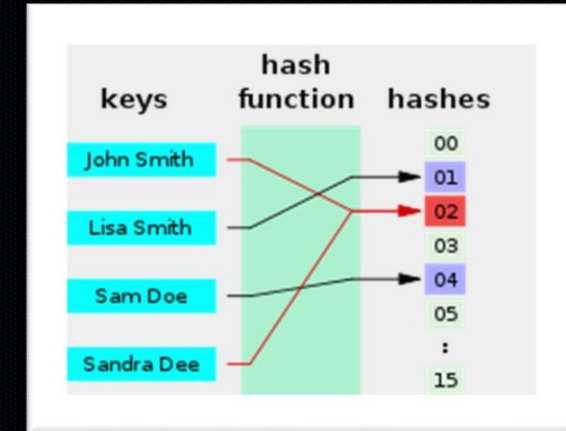
- Arquiteturas **estruturadas** e **não-estruturadas**

Arquitetura Peer-to-Peer estruturada

- A Rede de sobreposição é construída usando um mecanismo determinístico e “estruturado”.
- Mecanismo usado comumente: **DHT** (*Distributed Hash Table*).
- Dados e nós recebem uma chave aleatória (128~160 bits).

Desafio: **Dada uma chave de um dado, mapear para o identificador de um nó.**

- Neste caso, a requisição é roteada entre os nós até alcançar o nó com o dado solicitado.

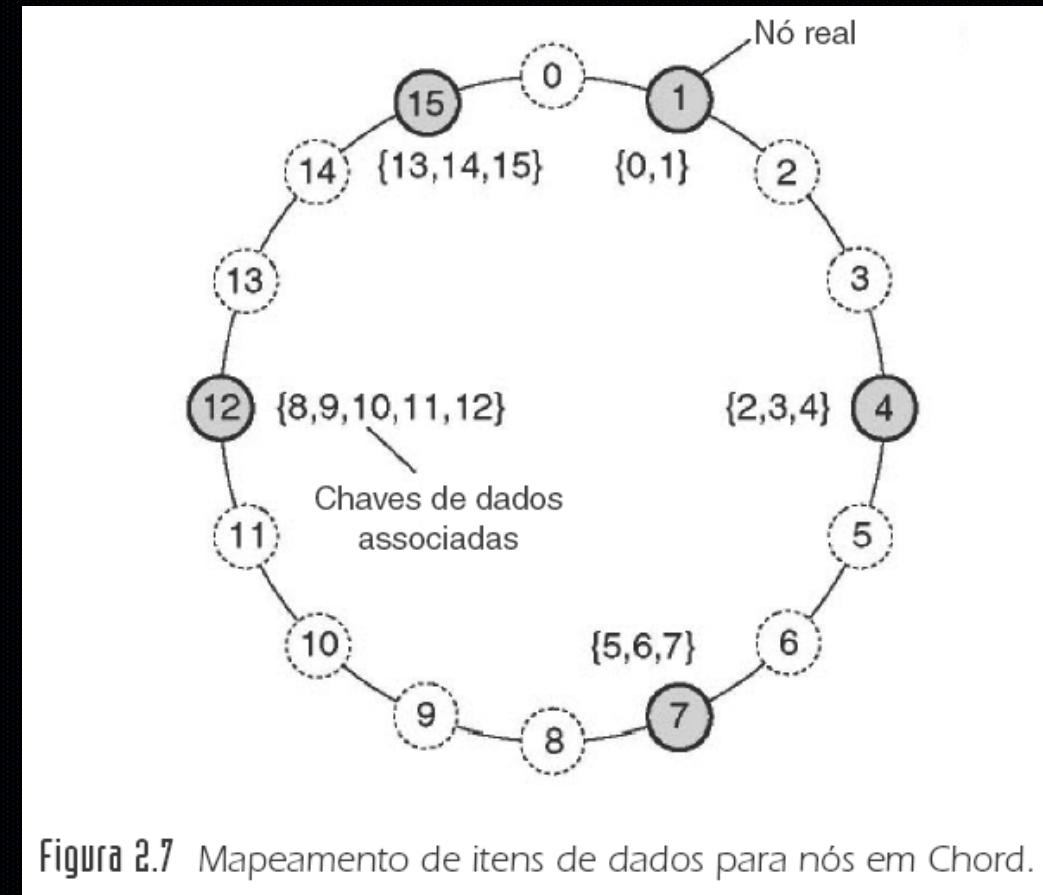


Arquitetura Peer-to-Peer estruturada

Chord: implementação de um DHT para redes Peer-to-Peer.

Nós estão logicamente organizados em um anel.

Cada nó recebe um identificador aleatório não ambíguo (semelhante a um RG).



Arquitetura Peer-to-Peer Não-estruturada

- Algoritmos aleatórios são utilizados para construir a rede de sobreposição.
- Cada nó mantém uma lista de nós vizinhos.
- Dados são também armazenados aleatoriamente.
- Como é realizada a busca?
 - Inunda toda a rede. Tempestade de broadcast.

Arquitetura Peer-to-Peer: Superpares

- À medida que a rede cresce, localizar itens de dados em sistemas P2P não estruturados pode ser problemático.
- Nós “diretórios” e/ou intermediários.
- Sempre que um nó comum se junta à rede, se liga a um dos superpares.
- Problema: Seleção do líder.

Arquitetura Peer-to-Peer: Superpares

Os superpares devem ter vida longa e alta disponibilidade.

+ Bateria
+ REDE
+ PROCESS

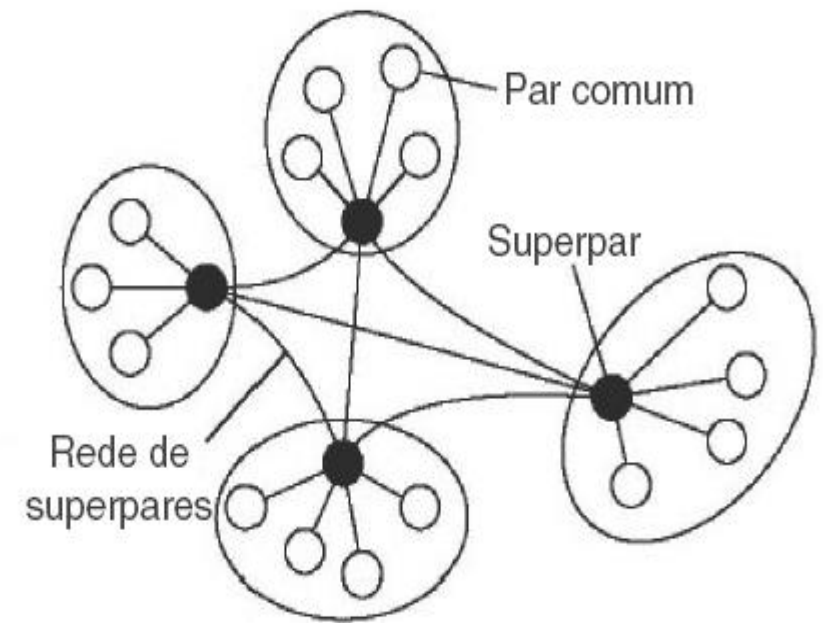


Figura 2.12 Organização hierárquica de nós em uma rede de superpares.

Arquiteturas híbridas

Soluções de cliente-servidor são combinadas com a forma de funcionamento da arquitetura descentralizada.

Híbrida = centralizada + descentralizada

Centralizada: Servir de diretório.

Descentralizada: Distribuição do conteúdo. Ex.: Skype, BitTorrent, WhatsApp.

Arquiteturas Híbridas

Sistemas distribuídos colaborativos

- Inicialmente, um esquema de procura pelo cliente-servidor tradicional.
- Subsequentemente, o nó junta-se para dar um mecanismo descentralizado de colaboração.

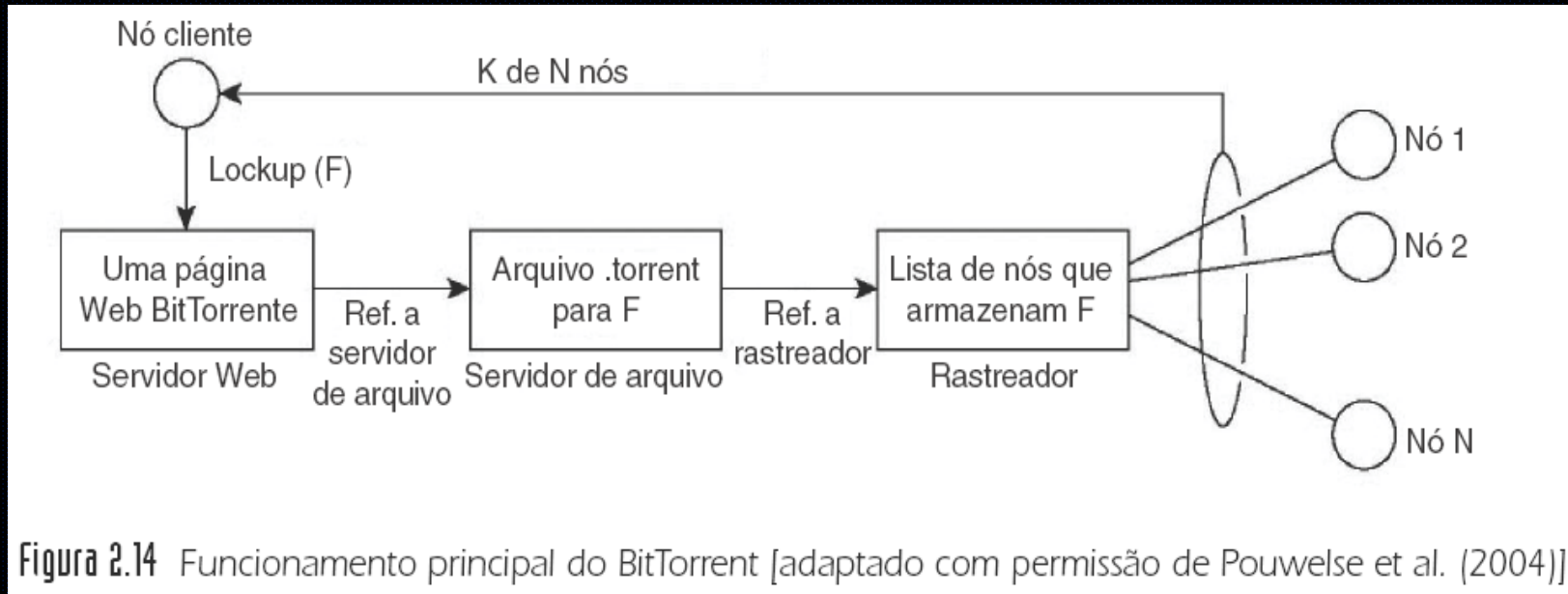


Figura 2.14 Funcionamento principal do BitTorrent [adaptado com permissão de Pouwelse et al. (2004)].

Arquiteturas Comparação

Arquitetura Centralizada

- Um servidor central controla as entradas e saídas de peers da rede
- Os peers registram no servidor central os recursos que compartilharão na rede
- Pesquisas por recursos disponíveis nos peers são efetuadas pelo servidor central
- O acesso aos recursos é feito diretamente entre peers
- Exemplos: Napster; eMule

Arquitetura Descentralizada

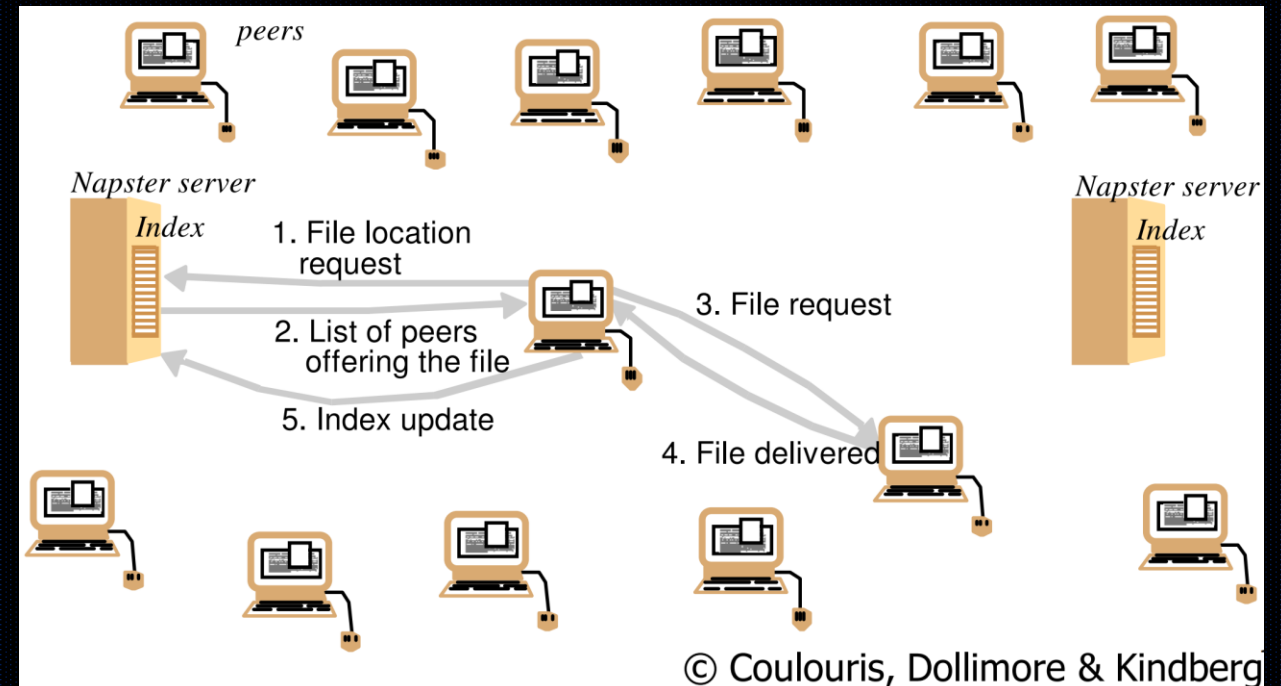
- Não há um elemento central
- Todos os nós possuem papel equivalente
- As pesquisas por recursos compartilhados são feitas por inundação (flooding)
- Gera um alto tráfego na rede
- Desempenho das pesquisas é ruim devido à necessidade de contactar muitos nós e aguardar a resposta
- Exemplos: Gnutella e JXTA

Arquitetura Híbrida

- Supernós permitem o ingresso dos nós na rede, podendo também exercer atividades de coordenação do funcionamento da rede, indexar os recursos compartilhados pelos nós e permitir a busca por estes recursos.
- Após localizado, um recurso pode ser obtido a partir da interação direta entre nós.
- Falha de um supernó pode ser tolerada elegendo dinamicamente outro supernó.
- Exemplos: Kazzaa; Skype.

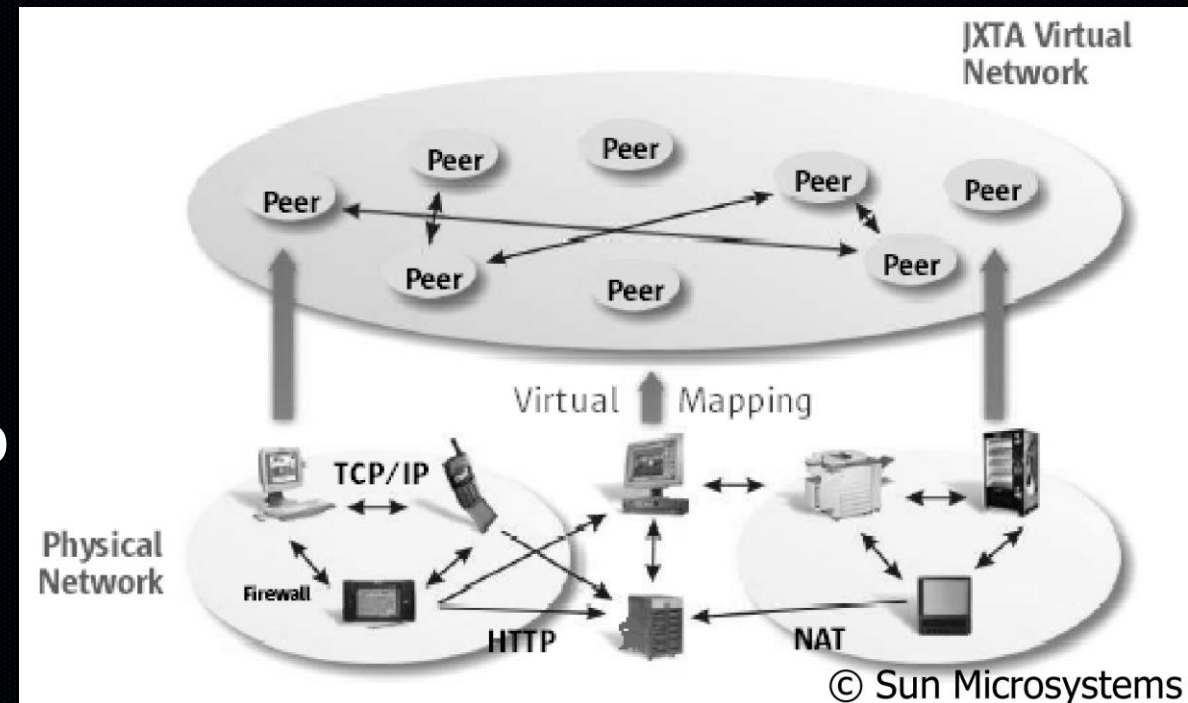
Napster

- Criado em 1999
- Utilizava um servidor central (replicado) para efetuar a procura de arquivos na rede
- Transferência de arquivos era feita diretamente entre os peers
- Sem o servidor central, que foi fechado por ordem judicial, a rede deixou de funcionar



JXTA

- Proposto pela Sun
- ~~Provê uma infra~~ Provê uma infra-estrutura simples de rede P2P estrutura simples de rede P2P sobre a qual podem ser criadas aplicações que empregam este paradigma de comunicação.
- Cria grupos de pares com interesses comuns.
- Mensagens JXTA são codificadas em XML.
- Permite estabelecimento de conexões seguras.
- Possui uma implementação padrão em Java.
- Padroniza os pares.



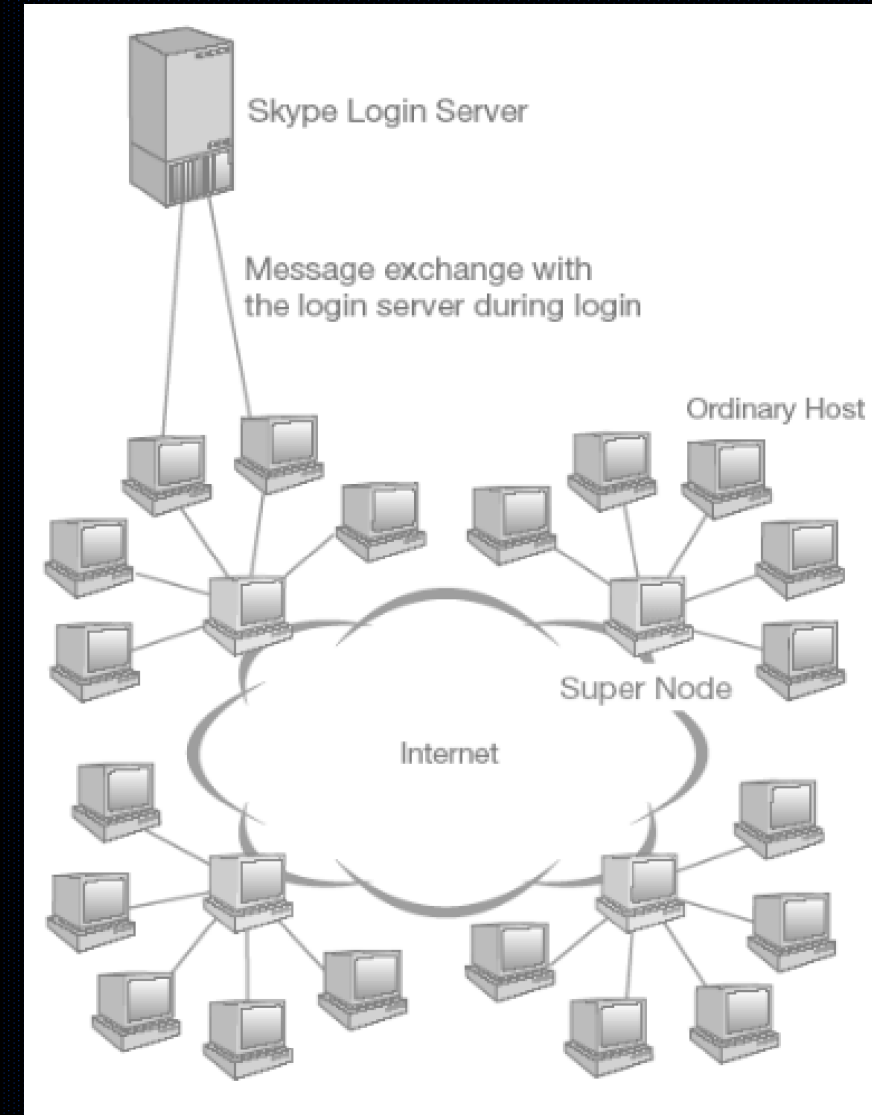
KAZAA / SKYPE

KAZAA

- Utiliza supernós para acesso à rede e para busca e indexação de conteúdo.

SKYPE

- Permite comunicação por áudio ou vídeo entre usuários.
- Utiliza supernós e um servidor de login.

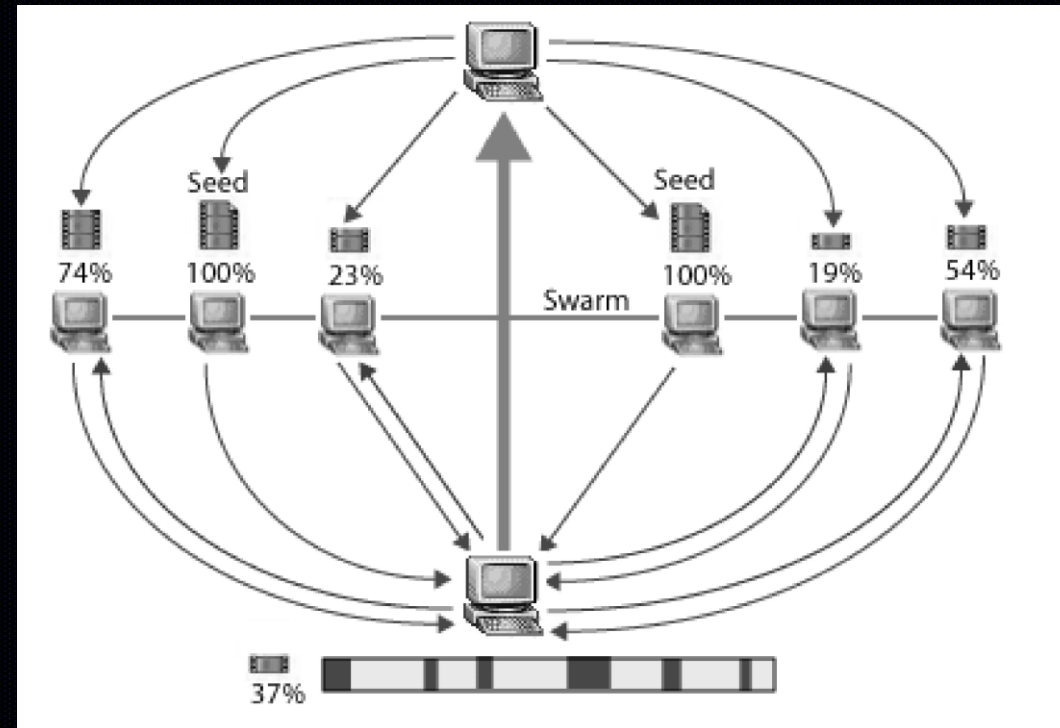


BitTorrent

- Protocolo P2P para download de arquivos.
- Há vários aplicativos clientes compatíveis.
- O arquivo compartilhado é dividido em vários pedaços, que são baixados simultaneamente.
- Pedaços recebidos já são compartilhados.
- O protocolo engloba mecanismos para:
 - Controlar a integridade dos pedaços de arquivos baixados
 - Recompensar quem compartilha arquivos, e penalizar quem tira proveito e não colabora

BitTorrent

- Arquivo .torrent
 - Criado por nó seed, que compartilha arquivo
 - Contém metadados que descrevem o que descrevem arquivo e permitem verificar sua integridade
 - Indica servidores *tracker*
- Servidores *tracker*
 - Coordenam a distribuição de arquivos
 - Indicam os nós para download
 - Controlam a velocidade do download
 - Se o nó não colabora (leech), perde banda



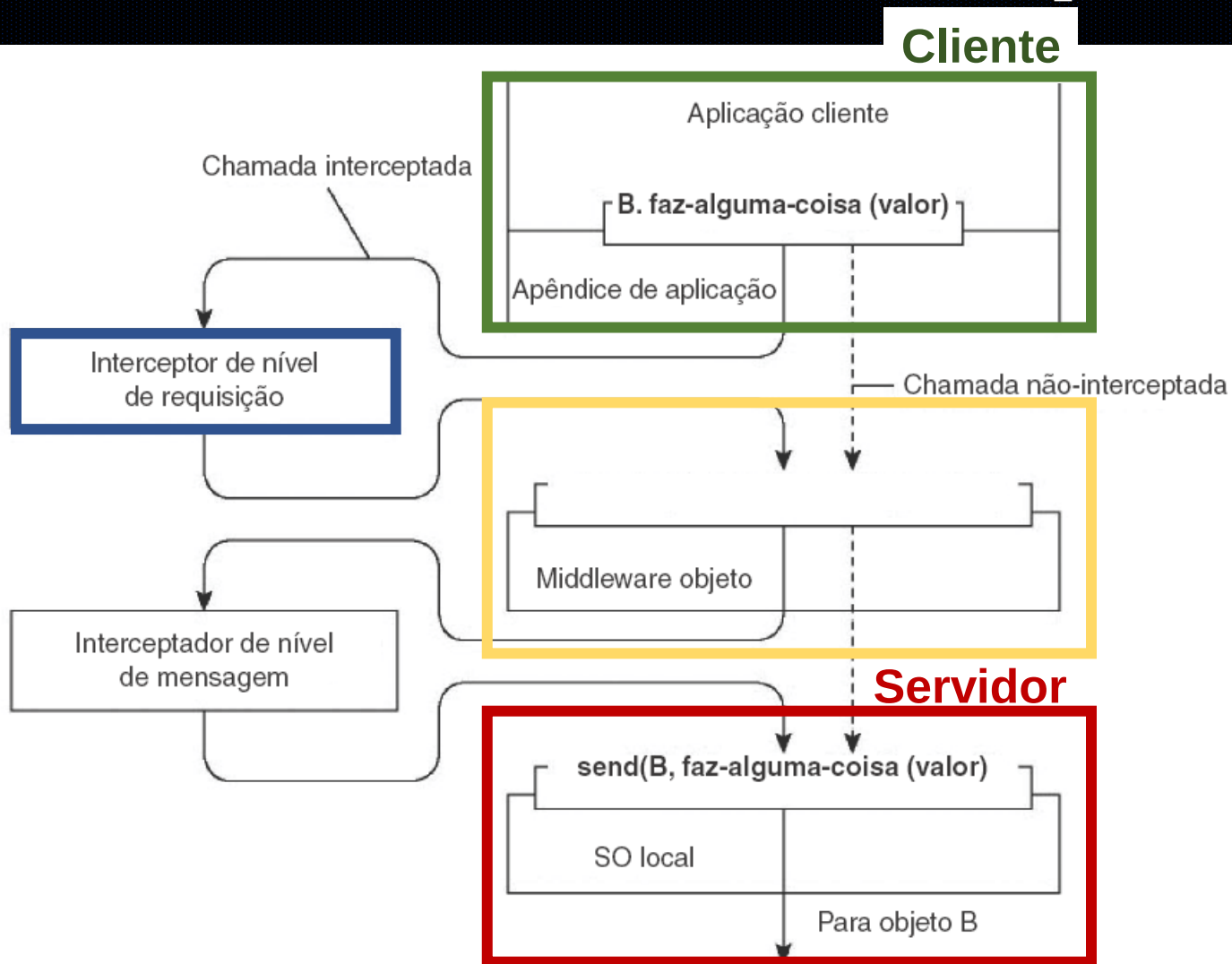
Arquiteturas vs. Middleware

- Podem ser estático ou dinâmico
 - Ocorrem em tempo de execução
 - Ocorrem na inicialização / instanciação
- E.g. Os middlewares como CORBA eram todos baseados em objetos com o objetivo de eliminar os problemas típicos em sistemas heterogêneos
 - Independência de Linguagem
 - Independência de arquitetura de hardware
 - Independência de sistema operacional
 - Independência de protocolo/transporte
- É simples e específico para aplicações com objetos remotos, porém, não é trivial criar ou estender para outros tipos de aplicações.
- Necessidade de **reconfiguração** dinâmica em tempo de execução.

Interceptadores

- Uma solução para reconfigurar a necessidade da aplicação?
Interceptadores.
- É uma construção de software que interromperá o fluxo de controle usual de execução.
- Em uma chamada desvia para outra chamada.
- **Em tempo de execução.**

Interceptadores

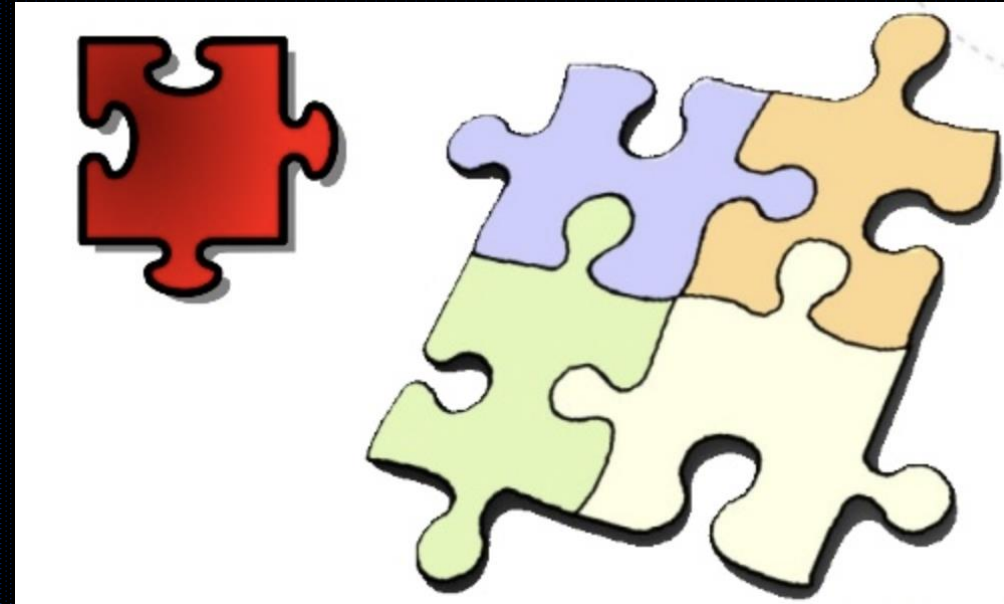


- Software que fica entre a **Aplicação** e o **Servidor**
- Verificar pré-requisitos na chamada
 - Memória
 - Banda
 - Processamento
- Reduz a performance do computador

Figura 2.15 Utilização de interceptadores para manipular invocações de objeto remoto.

Middleware Adaptativos

- As instâncias dos componentes são adicionadas e removidas conforme solicitadas em tempo de execução.
 - Módulos de Wifi
 - Módulos de 4g/5g
- Economia de memória para sistemas embarcados e ambientes IoT.
- A aplicação é “*tailored/personalizado*” de acordo com o contexto.
- Facilita criar aplicações sensíveis ao contexto.
- Preço é o gasto em monitoração



Concluindo

Foram abordados nesta aula:

- **Tipos de arquitetura**

Estes slides estão baseados na bibliografia

Sistemas Distribuídos de TANENBAUM, A. S. e STEEN, M. V. (2ª edição)

Na próxima aula:

- **Processos Distribuídos**

AULA Nº 04

SISTEMAS DISTRIBUÍDOS

Tipos de arquitetura

Os slides são baseados no livro de Sistemas Distribuídos de TANENBAUM, A. S. e STEEN, M. V.